

MedVisionAI

Project Team

Abdur Rehman Khan Durrani	22I-1903
Syed Abdul Wasay	22I-2037
Emad Hasan	22I-0453

Session 2022-2026

Supervised by

Ms. Kanza Hamid



Department of AI and Data Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2026

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Module 1 — Dataset & Ingestion Pipeline	2
1.3.2	Module 2 — Radiology & Image Processing	2
1.3.3	Module 3 — Retrieval & RAG Q&A Engine	3
1.3.4	Module 4 — Adaptive Flashcards & Quiz Engine	3
1.3.5	Module 5 — Web Application (React + Flask)	3
1.3.6	Module 6 — Evaluation & Dashboard	3
1.4	User Classes and Characteristics	3
2	Project Requirements	5
2.1	Use-case / Event Response / Storyboarding	5
2.2	Functional Requirements	7
2.2.1	Module 1 — Dataset & Ingestion Pipeline	7
2.2.2	Module 2 — Radiology & Image Processing	7
2.2.3	Module 3 — Retrieval & RAG Q&A Engine	7
2.2.4	Module 4 — Adaptive Flashcards & Quiz Engine	8
2.2.5	Module 5 — Web Application (React + Flask)	8
2.2.6	Module 6 — Evaluation & Dashboard	8
2.3	Non-Functional Requirements	8
2.3.1	Reliability	8
2.3.2	Usability	9
2.3.3	Performance	9
2.3.4	Security	9
2.4	Domain Model	9
3	System Overview	11
3.1	General Description	11
3.2	Architectural Design	11
3.2.1	System Architecture Diagram	12
3.3	Design Models	13

3.3.1	Activity Diagram	13
3.3.2	Class Diagram	14
3.3.3	Class-Level Sequence Diagram	14
3.3.4	State Transition Diagram	15
3.4	Data Design	16
3.4.1	Data Storage	16
3.4.2	Data Entities	16
3.4.3	Data Processing Flow	17
3.4.4	Data Flow Diagram	17
4	Implementation and Testing [UPTO THE CURRENT ITERATION ONLY]	19
4.1	Algorithm Design	20
4.2	External APIs/SDKs	23
4.3	Testing Details	23
4.3.1	Unit Testing	24
	References	25
A	Appendices	27
A.1	Appendix A	28
A.1.1	Use Case Diagram (MedVisionAI)	28
A.1.2	Detail Use Case	29
A.1.3	Event-Response Table for MedVisionAI	30
A.1.4	Story Board Example For MedVisionAI	31
A.2	Appendix B	32
A.2.1	Domain Model for MedVisionAI	32
A.3	Appendix C	33
A.3.1	Box And Line For MedVisionAI	33
A.3.2	Architecture Pattern for MedVisionAI	34
A.4	Appendix D	35
A.4.1	Activity Diagram	35
A.4.2	Class Diagram	36
A.4.3	Sequence Diagram	37
A.4.4	State Transition Diagram	38
A.4.5	Data Flow Diagram	38

List of Figures

2.1	Use-case diagram for MedVisionAI	6
2.2	Domain model of the MedVisionAI system	10
3.1	High-level system architecture of MedVisionAI	12
3.2	Activity diagram of MedVisionAI workflow	13
3.3	Class diagram representing MedVisionAI entities	14
3.4	Class-level sequence diagram for multimodal query flow	15
3.5	State transition diagram of MedVisionAI	16
3.6	Data flow diagram for MedVisionAI	17
4.1	Algorithm Design of RAG	20
4.2	Algorithm Design of SM-2	21
4.3	Algorithm Design of Anonymizer	22
4.4	Algorithm Design of Quiz	22
4.5	Algorithm Design of Grad-Cam	23
4.6	Unit Testing for MedVisionAI	24
A.1	Use Case Diagram for MedVisionAI	28
A.2	Detail Use Case	29
A.3	Event Response Table	30
A.4	Story Board	31
A.5	Domain Model	32
A.6	Box and Line Diagram For MedVisionAI	33
A.7	Architecture Pattern For MedVisionAI	34
A.8	Activity Diagram For MedVisionAI	35
A.9	Class Diagram For MedVisionAI	36
A.10	Sequence Diagram For MedVisionAI	37
A.11	State Transition Diagram For Online Shopping Application	38

List of Tables

Chapter 1

Introduction

The purpose of this document is to describe the requirements, design scope, and modular structure of **MedVisionAI**, an educational assistant aimed at supporting medical students in learning *chest X-ray interpretation*. Traditional radiology training is resource-intensive, requiring expert supervision and large volumes of annotated data [3, 7]. In developing regions, medical students face limited access to expert guidance, case diversity, and interactive learning resources.

This project addresses these challenges by proposing a lightweight, retrieval-augmented generation (RAG) system that integrates multimodal search, small LLM reasoning, adaptive flashcards, and explainability overlays. The system is strictly educational, ensuring that students gain confidence and deeper understanding of chest X-ray patterns without making clinical or diagnostic claims.

1.1 Problem Statement

Medical students and early-career clinicians often lack access to scalable, interactive platforms that simulate the process of interpreting chest X-rays [7]. Existing learning methods rely heavily on static slides, textbooks, or paid learning resources, which do not provide adaptive or interactive feedback. Moreover, publicly available datasets exist but are underutilized in educational contexts [3], and most AI-based solutions are either too large to run on modest hardware or focus solely on automated diagnosis rather than teaching [1, 2].

This project addresses these limitations by developing a lightweight educational assistant that:

- Provides multimodal retrieval (text queries + chest X-rays) for relevant educational material [4, 5].

- Supports small, locally adaptable LLMs fine-tuned with Low-Rank Adaptation (LoRA) for efficiency [2].
- Integrates adaptive flashcards using the SM-2 spaced repetition algorithm.
- Offers explainability features such as saliency overlays to help students understand AI reasoning [6].

1.2 Scope

The scope of **MedVisionAI** is to develop a privacy-preserving, educational Chest X-ray (CXR) learning assistant that integrates OCR ingestion of lecture slides, multimodal retrieval, a retrieval-augmented generation (RAG) Q&A engine using a compact LoRA-adapted LLM, and an adaptive flashcard/quiz engine (SM-2). The system will focus exclusively on frontal chest X-rays (AP/PA views), using publicly available, de-identified datasets (e.g., NIH ChestX-ray8, MIMIC-CXR, CheXpert) to build searchable, evidence-grounded explanations and study aids.

The project will emphasize low-resource deployment (e.g., on RTX 3050 GPUs or Google Colab), reproducibility through Docker, and a modular pipeline for ingestion, retrieval, and evaluation. Out of scope are modalities such as CT, MRI, and ultrasound, as well as any clinical diagnostic use or hospital system integration.

1.3 Modules

The system consists of six key modules, each handling a distinct aspect of the workflow:

1.3.1 Module 1 — Dataset & Ingestion Pipeline

Scripts to accept lecture slides (PDF/PPTX), instructor notes, and de-identified CXR images. It performs OCR using Tesseract, text normalization, segmentation, and embedding generation with SentenceTransformers and CLIP. **Deliverable:** Ingestion scripts and FAISS index.

1.3.2 Module 2 — Radiology & Image Processing

Handles local anonymization using OpenCV, lightweight captioning with CLIP, and explainability overlays (Grad-CAM, Integrated Gradients). **Deliverable:** Anonymizer, captioner, and overlay generator tools.

1.3.3 Module 3 — Retrieval & RAG Q&A Engine

Retrieves top-k relevant passages and captions using FAISS and uses a fine-tuned TinyL-LaMA with LoRA/QLoRA to generate contextual educational answers. **Deliverable:** Inference API for multimodal Q&A.

1.3.4 Module 4 — Adaptive Flashcards & Quiz Engine

Extracts Q/A pairs from lecture content and RAG outputs, applies SM-2 spaced repetition, and generates quizzes with adaptive difficulty. **Deliverable:** Flashcard database, scheduler, and quiz APIs.

1.3.5 Module 5 — Web Application (React + Flask)

Frontend (React + Material UI) and backend (Flask) handle uploads, Q&A, flashcards, and dashboards, packaged with Docker Compose. **Deliverable:** Integrated prototype and documentation.

1.3.6 Module 6 — Evaluation & Dashboard

Implements evaluation metrics for retrieval, generation, and image overlap, and includes a dashboard summarizing student performance. **Deliverable:** Evaluation report and dashboards.

1.4 User Classes and Characteristics

The primary users of **MedVisionAI** are:

- **Medical Students:** Core users who will interact with the system for adaptive radiology learning.
- **Medical Educators:** Faculty integrating MedVisionAI as a supplementary teaching tool.
- **Researchers:** Those studying lightweight educational RAG systems.
- **Developers:** The team maintaining system design, testing, and evaluation.

Chapter 2

Project Requirements

This chapter specifies the functional and non-functional requirements for the **MedVisionAI** system. These requirements are derived from the proposal document and informed by the literature on multimodal learning, retrieval-augmented generation (RAG), and explainable AI.

2.1 Use-case / Event Response / Storyboarding

Since **MedVisionAI** is an interactive web-based application intended for educational purposes, the most appropriate requirement-gathering technique is the **use-case approach**. Key actors include medical students, educators, and researchers. The system emphasizes graphical interaction, adaptive feedback, and explainable AI visualization; hence, a combination of use-case diagrams and storyboards effectively captures user interactions.

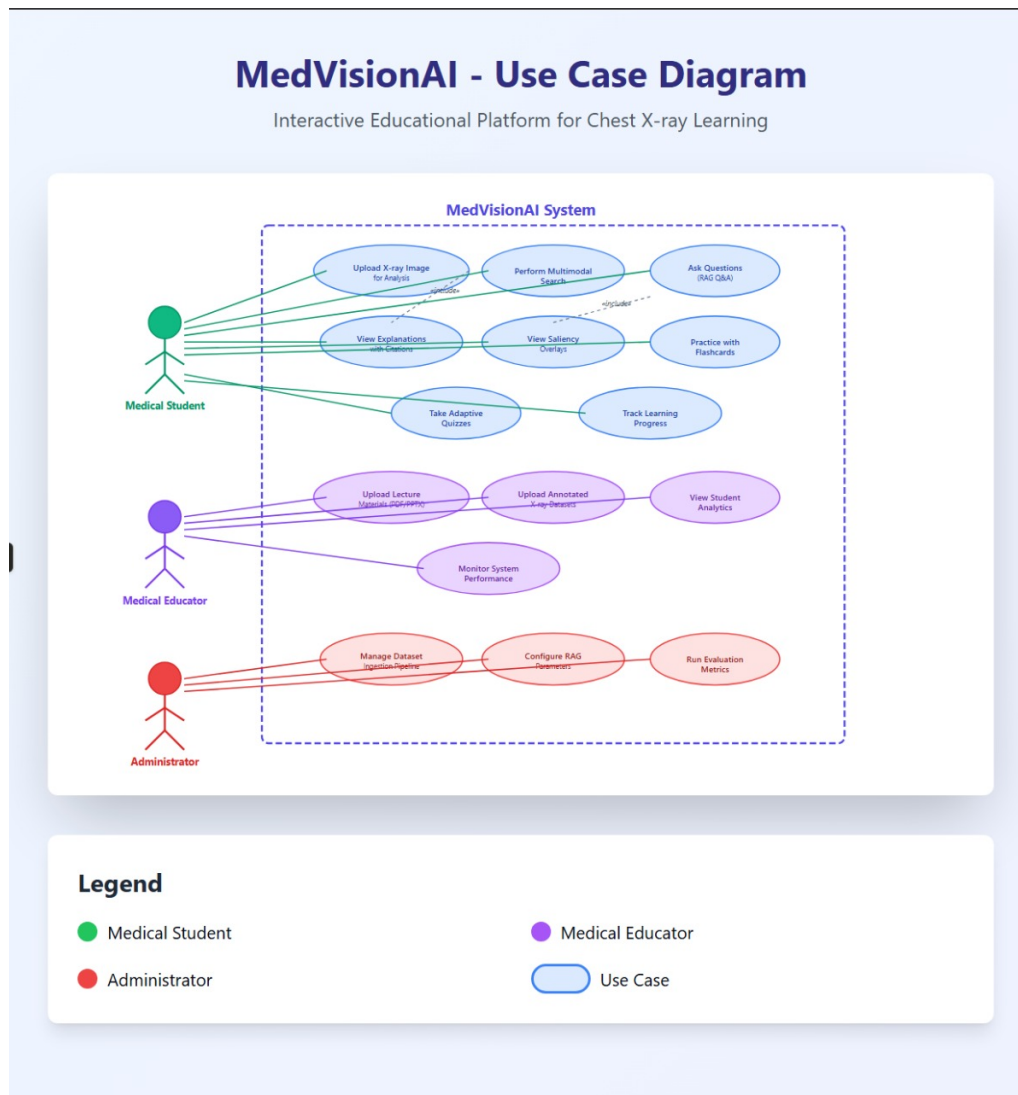


Figure 2.1: Use-case diagram for MedVisionAI

Major use cases include:

- Uploading lecture slides or de-identified chest X-ray datasets.
- Searching multimodal data (text queries + X-ray images) via RAG-based retrieval.
- Interacting with adaptive flashcards and quizzes for spaced repetition learning.
- Reviewing explainability overlays and saliency maps for visual reasoning.
- Tracking learning progress through evaluation dashboards.

2.2 Functional Requirements

The functional requirements define the essential behavior and services that the system must provide. They are categorized by modules, consistent with the project proposal.

2.2.1 Module 1 — Dataset & Ingestion Pipeline

1. The system shall allow users to upload lecture slides (PDF/PPTX), notes, and de-identified CXR images.
2. The ingestion pipeline shall extract text using Tesseract OCR and normalize content (header/footer removal, casing, token segmentation) [3, 7].
3. The system shall generate text embeddings using SentenceTransformers and image embeddings using CLIP/ViT-tiny [4].
4. The embeddings and metadata shall be stored locally in FAISS and SQLite for efficient retrieval [5].

2.2.2 Module 2 — Radiology & Image Processing

1. The system shall anonymize uploaded X-rays using OpenCV-based text detection and masking.
2. The system shall generate lightweight image captions using CLIP-based methods.
3. The system shall provide explainability overlays such as Grad-CAM and Integrated Gradients for model transparency [6].

2.2.3 Module 3 — Retrieval & RAG Q&A Engine

1. The retrieval component shall return top-k relevant passages and captions for a given multimodal query using FAISS.
2. The RAG engine shall assemble retrieved content and use TinyLLaMA adapted via LoRA and QLoRA for efficient fine-tuning [1, 2].
3. Each generated answer shall include contextual evidence and citation references.

2.2.4 Module 4 — Adaptive Flashcards & Quiz Engine

1. The system shall extract Q/A pairs from lecture slides and validated RAG responses.
2. The flashcard engine shall implement the SM-2 spaced repetition algorithm to optimize retention.
3. The quiz generator shall adaptively adjust difficulty based on the student's performance.

2.2.5 Module 5 — Web Application (React + Flask)

1. The frontend shall be developed using React and Material UI to ensure a responsive and user-friendly interface.
2. The backend shall be implemented in Flask, supporting endpoints for ingestion, retrieval, and flashcard modules.
3. The system shall support containerized deployment using Docker Compose.

2.2.6 Module 6 — Evaluation & Dashboard

1. The system shall compute evaluation metrics including Recall@k, MRR, ROUGE, and BERTScore for generated outputs.
2. The dashboard shall visualize student performance, retrieval quality, and explainability overlap metrics.
3. The evaluation process shall also support formative user studies to assess educational effectiveness.

2.3 Non-Functional Requirements

The non-functional requirements define the quality attributes that ensure **MedVisionAI** meets performance, reliability, and usability standards.

2.3.1 Reliability

- The system shall maintain a mean time between failures (MTBF) of at least 500 hours during testing.

- All key modules shall include logging and exception handling to ensure traceability.

2.3.2 Usability

- The interface shall be intuitive and support accessible interactions for diverse learners.
- Users shall be able to complete any learning task (e.g., quiz, flashcard review) within three steps.

2.3.3 Performance

- At least 90% of retrieval queries shall execute within two seconds for datasets up to 50,000 embeddings.
- Generated responses from the LLM shall appear within five seconds on an RTX 3050 GPU or equivalent environment.

2.3.4 Security

- All uploaded data shall be de-identified and stored in a secure, isolated local environment.
- The anonymization module shall automatically remove any burned-in text or identifiers from images [7].

2.4 Domain Model

The domain model represents core entities such as users, datasets, embeddings, flashcards, and results. It highlights the relationships among ingestion, retrieval, and learning modules.

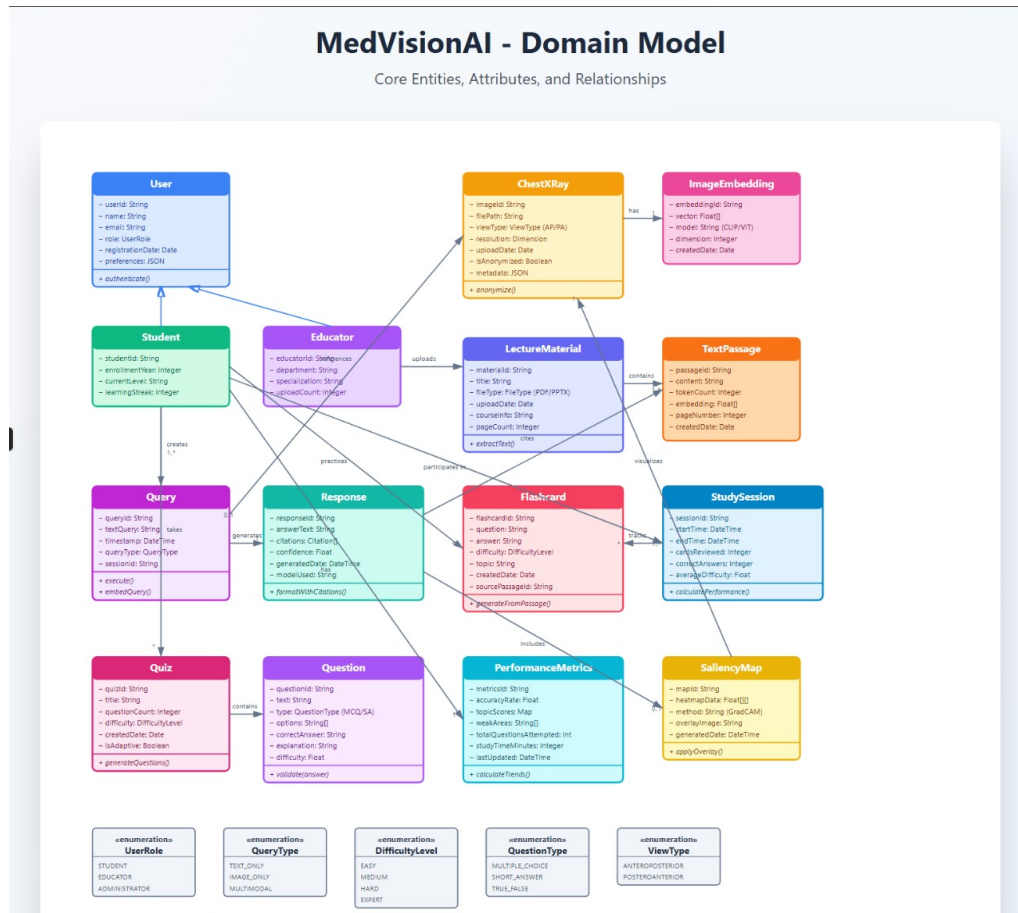


Figure 2.2: Domain model of the MedVisionAI system

Chapter 3

System Overview

3.1 General Description

MedVisionAI is a multimodal educational platform that enables medical students to interactively learn chest X-ray interpretation through AI-assisted tools. The system integrates text and image retrieval, question-answering, adaptive flashcards, and explainability overlays to provide an engaging and effective learning experience.

The application adopts a modular, data-driven design based on retrieval-augmented generation (RAG) and lightweight fine-tuning methods such as LoRA and QLoRA [1, 2]. It leverages open datasets like NIH ChestX-ray8 and MIMIC-CXR [3, 7], along with modern multimodal frameworks such as CLIP [4] and FAISS [5], ensuring reproducibility and accessibility for users with limited hardware resources.

At a high level, the system ingests medical educational materials (slides, notes, and de-identified X-ray images), processes them into text and image embeddings, and uses these representations for retrieval and educational question answering. The web interface, built on a React–Flask architecture, allows users to query data, review results, study via flashcards, and visualize AI explanations.

3.2 Architectural Design

The architecture of **MedVisionAI** follows a modular, multi-tier design combining client–server and layered architecture patterns. The major layers include:

- **Presentation Layer:** A web-based frontend built using React and Material UI for intuitive interaction and accessibility.

- **Application Layer:** Flask-based backend APIs manage user requests, data ingestion, retrieval, RAG question answering, and flashcard generation.
- **Processing Layer:** Handles the ingestion pipeline, embedding generation, multi-modal retrieval, and lightweight LLM reasoning.
- **Data Layer:** Manages persistent storage of embeddings, datasets, and metadata using FAISS and SQLite.

3.2.1 System Architecture Diagram

Figure 3.1 provides a high-level architectural overview of how these components interact.

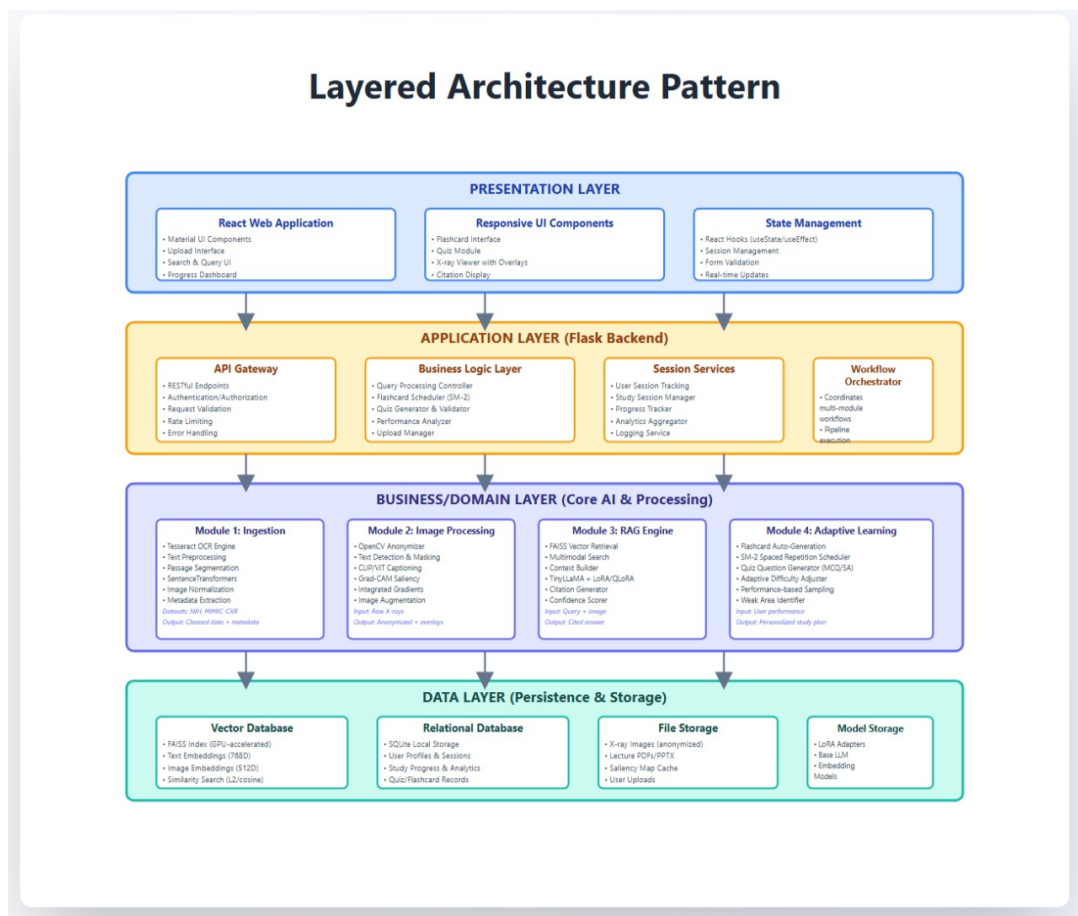


Figure 3.1: High-level system architecture of MedVisionAI

The modules interact as follows:

1. The **Dataset & Ingestion Pipeline** converts raw text and image data into embeddings stored in FAISS.

2. The **Retrieval & RAG Engine** retrieves relevant multimodal content and generates contextual responses using a fine-tuned TinyLLaMA model.
3. The **Radiology & Image Processing** module supports anonymization, captioning, and explainability overlays such as Grad-CAM [6].
4. The **Flashcard & Quiz Engine** converts validated Q/A data into adaptive learning materials using SM-2 spaced repetition.
5. The **Evaluation & Dashboard** layer visualizes system performance and user learning outcomes.

This modular decomposition ensures scalability, maintainability, and clear separation of concerns while supporting easy deployment using Docker Compose.

3.3 Design Models

The system employs an **object-oriented design** approach. Various UML and system-level diagrams describe component interactions and data flow.

3.3.1 Activity Diagram

The activity diagram illustrates the workflow from data ingestion to user interaction.

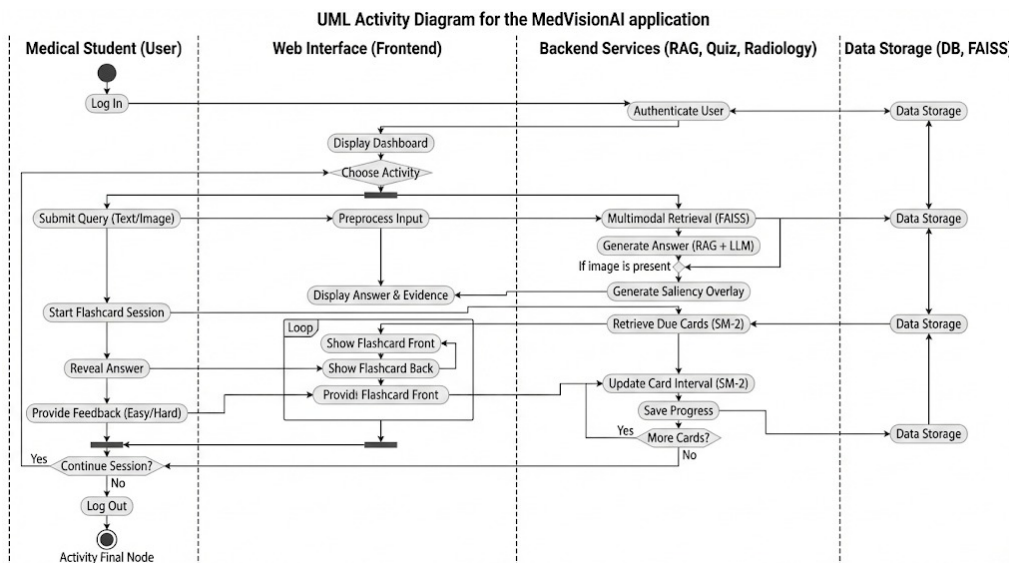


Figure 3.2: Activity diagram of MedVisionAI workflow

3.3.2 Class Diagram

The class diagram models relationships among the main entities such as Dataset, Embedding, User, Flashcard, and Evaluation.

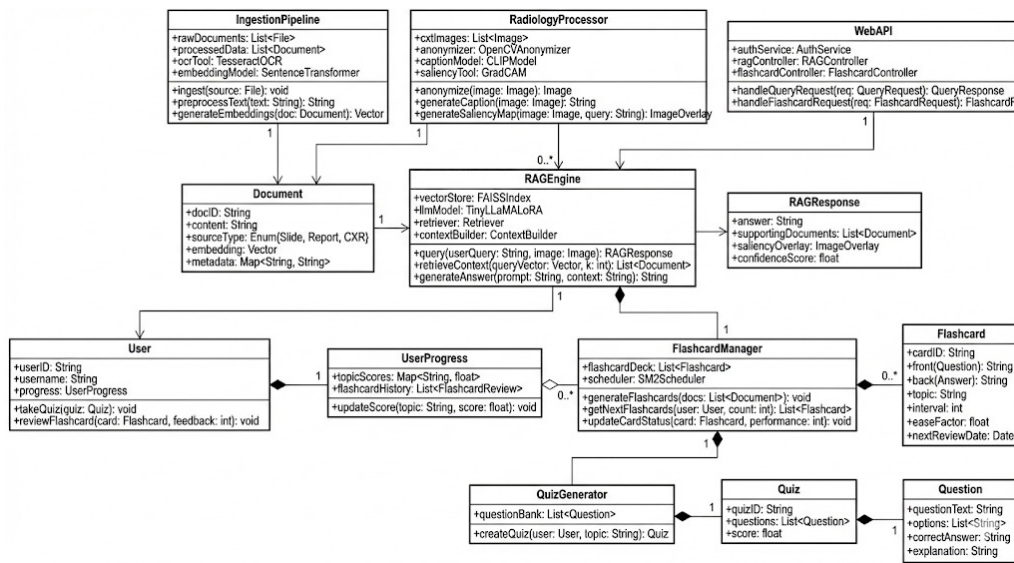


Figure 3.3: Class diagram representing MedVisionAI entities

3.3.3 Class-Level Sequence Diagram

The class-level sequence diagram highlights the flow between the user, frontend, backend APIs, and the RAG retrieval engine during a question-answer session.

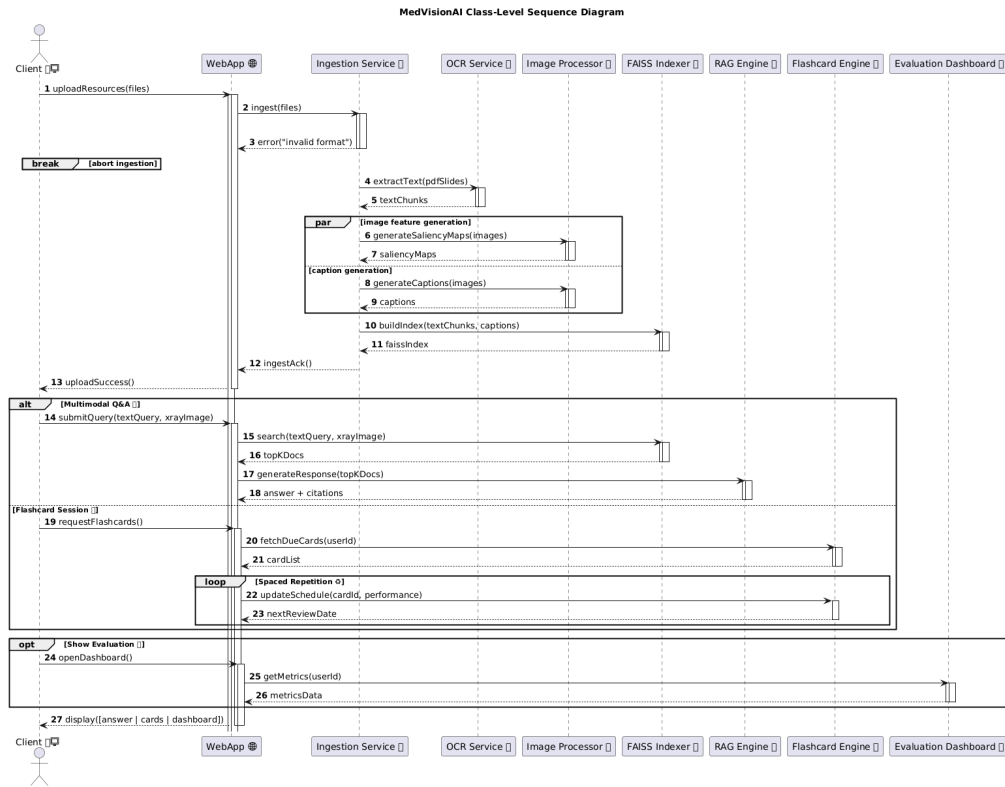


Figure 3.4: Class-level sequence diagram for multimodal query flow

3.3.4 State Transition Diagram

This diagram models the transitions between system states such as Data Ingestion, Index Ready, Query Processing, Flashcard Generation, and Evaluation.

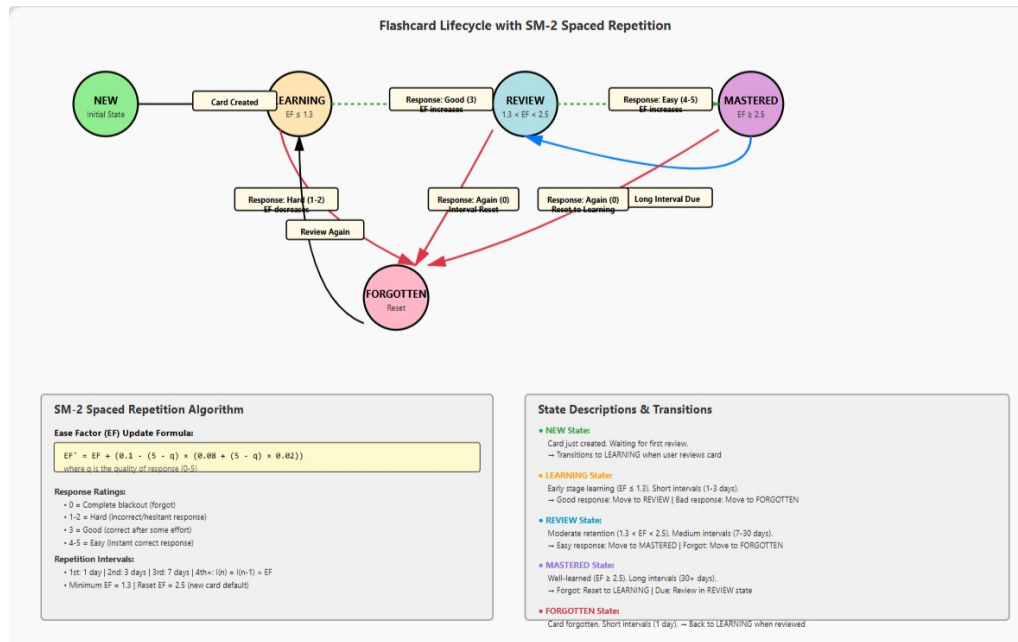


Figure 3.5: State transition diagram of MedVisionAI

3.4 Data Design

MedVisionAI uses both structured and unstructured data components to handle multi-modal educational content efficiently.

3.4.1 Data Storage

- **FAISS Index:** Stores dense vector embeddings for fast similarity search in both text and image domains [5].
- **SQLite Database:** Maintains metadata, user information, and study progress records.
- **Local JSON Files:** Used for lightweight configuration storage (e.g., quiz performance, scheduler state).

3.4.2 Data Entities

Key data entities in the system include:

- **Document:** Text segments extracted from lecture slides via OCR.
- **Image:** De-identified chest X-rays with associated embeddings and captions.

- **Embedding:** Vector representations generated using SentenceTransformers and CLIP.
- **Flashcard:** Adaptive learning unit storing question-answer pairs and spaced repetition metrics.
- **Evaluation:** Stores retrieval and generation scores such as Recall@k, ROUGE, and BERTScore.

3.4.3 Data Processing Flow

All uploaded data undergo a standardized process:

1. Preprocessing (OCR, normalization, and anonymization).
2. Embedding generation and FAISS indexing.
3. Retrieval and RAG reasoning during user queries.
4. Adaptive feedback loop using flashcard updates and performance tracking.

This design ensures a consistent, efficient, and explainable data lifecycle throughout the MedVisionAI system.

3.4.4 Data Flow Diagram

The data flow diagram (DFD) illustrates how information moves through the MedVisionAI system, from raw data ingestion to retrieval-driven learning interactions.

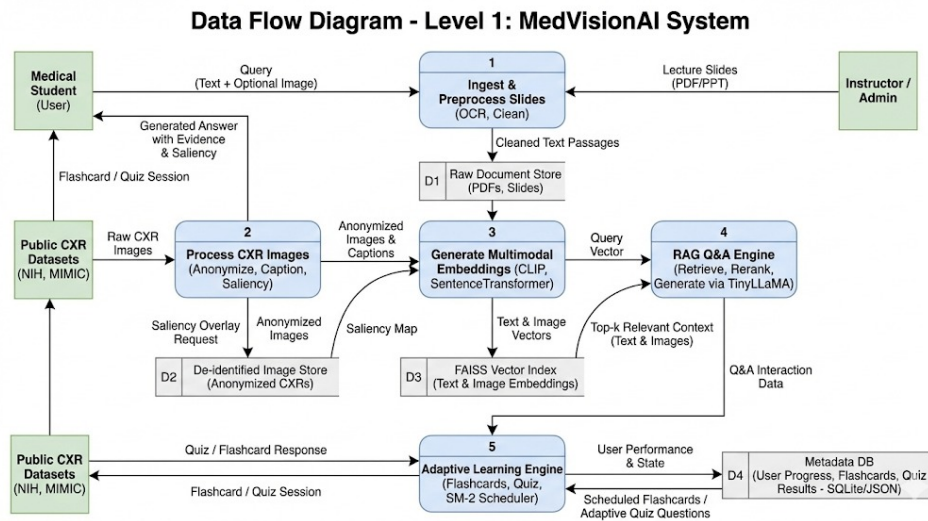


Figure 3.6: Data flow diagram for MedVisionAI

Chapter 4

Implementation and Testing [UPTO THE CURRENT ITERATION ONLY]

The MedVisionAI system is designed as an intelligent, multimodal educational assistant that helps medical students develop strong diagnostic skills in chest X-ray (CXR) interpretation. The project integrates modern machine learning techniques—specifically vision-language retrieval, lightweight large-language-model inference, and adaptive learning algorithms—to create an interactive environment where students can practice clinical reasoning with real-time feedback.

The system operates by combining visual information from student provided X-ray images with textual questions, enabling context-aware reasoning through a retrieval-augmented generation (RAG) pipeline. To support the learning process, the platform includes features such as automated anonymization of patient identifiers, Grad-CAM-based explainability overlays, adaptive flashcards built using the SM-2 spaced repetition algorithm, and an intelligent quiz generator that prioritizes weak areas for each learner.

MedVisionAI is built with a modular and data-driven architecture. Each subsystem image processing, embedding retrieval, LLM reasoning, explainable AI, and adaptive assessment functions as an independent component that integrates seamlessly into the main pipeline. This modular design ensures scalability, maintainability, and the flexibility to incorporate updated models or additional diagnostic modalities in the future.

From a contextual perspective, the project addresses a major challenge in medical education: the difficulty of gaining hands-on experience with large, diverse datasets of radiographic images. By leveraging open-source models and efficient inference techniques, the system provides a low-resource solution that runs effectively on consumer GPU hardware while maintaining strict privacy and data-protection standards. The platform empowers medical learners to explore real clinical scenarios in a safe, guided, and interactive manner.

4.1 Algorithm Design

The MedVisionAI system integrates several core algorithms to deliver accurate, explainable, and adaptive medical learning functionality. The Multimodal RAG pipeline combines visual embeddings from the chest X-ray with textual embeddings from the user's question to retrieve relevant medical knowledge before generating context-aware answers. The SM-2 spaced repetition model powers the flashcard module, optimizing review intervals based on learner performance. A privacy-preserving anonymization algorithm detects and blurs burned-in patient identifiers using MSER region detection and localized Gaussian blurring. For explainability, the Grad-CAM model produces saliency heatmaps that highlight the regions of the X-ray influencing the model's reasoning. Lastly, the adaptive quiz sampling algorithm evaluates student performance history to dynamically generate quizzes that focus on weak areas, ensuring targeted and effective learning. Following are few examples of algorithms/pseudocode.

Examples:

```

ALGORITHM MedVision_RAG_Inference
INPUT: User_Image (I), User_Question (Q)
OUTPUT: Generated_Answer (A), Saliency_Map (M), Evidence_Docs (D)
|
1. // Step 1: Embedding Generation
2. img_embedding ← CLIP_Vision_Model.encode(I)
3. txt_embedding ← SentenceTransformer.encode(Q)
4. combined_query ← normalize(img_embedding + txt_embedding)

5. // Step 2: Semantic Retrieval
6. relevant_indices ← FAISS_Index.search(combined_query, k=5)
7. context_text ← Database.fetch_passages(relevant_indices)

8. // Step 3: Contextual Prompting
9. system_prompt ← "You are a medical tutor. Use this context: " + context_text
10. full_prompt ← system_prompt + " Question: " + Q
11. Answer (A) ← TinyLLaMA_LoRA.generate(full_prompt)

12. // Step 4: Explainability Overlay
13. Saliency_Map (M) ← GradCAM(TinyLLaMA, I)

14. RETURN A, M, relevant_indices.metadata

```

Figure 4.1: Algorithm Design of RAG

```

ALGORITHM SM2_Scheduler
INPUT: User_Grade (G) [0-5], Previous_Interval (I_prev), Previous_Ease_Factor (EF_prev)
OUTPUT: Next_Interval (I_new), New_Ease_Factor (EF_new)

1. // Step 1: Calculate Interval based on performance
2. IF G >= 3 THEN // User answered correctly
3.     IF I_prev == 0 THEN
4.         I_new ← 1 // First interval is 1 day
5.     ELSE IF I_prev == 1 THEN
6.         I_new ← 6 // Second interval is 6 days
7.     ELSE
8.         I_new ← round(I_prev * EF_prev)
9.     END IF
10. ELSE // User failed
11.     I_new ← 1 // Reset interval to 1 day
12. END IF
13. // Step 2: Update Ease Factor (EF)
14. // Formula: EF' = EF + (0.1 - (5-G) * (0.08 + (5-G)*0.02))
15. delta ← (5 - G)
16. EF_new ← EF_prev + (0.1 - delta * (0.08 + delta * 0.02))
17. // Step 3: Enforce Minimum Ease Factor
18. IF EF_new < 1.3 THEN
19.     EF_new ← 1.3
20. END IF
21. RETURN I_new, EF_new

```

Figure 4.2: Algorithm Design of SM-2

```

ALGORITHM Image_Anonymization
INPUT: Raw_XRay_Image (Img)
OUTPUT: Deidentified_Image (Img_Clean)

1. // Step 1: Preprocessing
2. Img_Gray ← convert_to_grayscale(Img)
3. Img_Contrast ← adaptive_threshold(Img_Gray)

4. // Step 2: Text Region Detection
5. Text_Regions ← OpenCV.MSER_detect(Img_Contrast)
   // OR use a specific text detector like EAST/Tesseract

6. // Step 3: Masking
7. FOR EACH region (x, y, w, h) IN Text_Regions DO
8.     // Apply Gaussian Blur to the region
9.     Img_Clean[y:y+h, x:x+w] ← GaussianBlur(Img[y:y+h, x:x+w], kernel=(51,51))
10. END FOR

11. RETURN Img_Clean

```

Figure 4.3: Algorithm Design of Anonymizer

```

ALGORITHM Adaptive_Quiz_Generator
INPUT: User_History (H), Topic_List (T), Quiz_Size (N=10)
OUTPUT: Quiz_Questions (Q_List)

1. // Step 1: Calculate Topic Weights
2. Weights ← []
3. FOR EACH topic IN T DO
4.     Accuracy ← H.get_accuracy(topic)
5.     // Higher weight for lower accuracy (inverse relationship)
6.     Weight ← 1.0 - Accuracy + 0.1 // 0.1 buffer ensures non-zero chance
7.     Weights.append(Weight)
8. END FOR

9. // Step 2: Weighted Sampling
10. Probabilities ← normalize_sum_to_one(Weights)
11. Selected_Topics ← weighted_random_sample(T, weights=Probabilities, k=N)

12. // Step 3: Retrieve Questions
13. Q_List ← []
14. FOR EACH topic IN Selected_Topics DO
15.     Question ← DB.fetch_random_question(topic)
16.     Q_List.append(Question)
17. END FOR
18. RETURN Q_List

```

Figure 4.4: Algorithm Design of Quiz

```

ALGORITHM GradCAM_Generator
INPUT: Model (M), Input_Image (I), Target_Layer (L)
OUTPUT: Heatmap_Overlay (H)

1. // Step 1: Forward Pass
2. Output_Scores ← M.forward(I)
3. Target_Class ← argmax(Output_Scores)

4. // Step 2: Backward Pass (Gradients)
5. Gradients ← compute_gradients(Output_Scores[Target_Class], L.feature_maps)
6. Pooled_Gradients ← global_average_pool(Gradients)

7. // Step 3: Weighted Combination
8. // Multiply feature maps by pooled gradients
9. Weighted_Maps ← L.feature_maps * Pooled_Gradients
10. Heatmap_Raw ← ReLU(sum(Weighted_Maps))

11. // Step 4: Normalization & Overlay
12. Heatmap_Norm ← min_max_scale(Heatmap_Raw, 0, 1)
13. Heatmap_Color ← apply_colormap(Heatmap_Norm, JET)
14. H ← blend(I, Heatmap_Color, alpha=0.5)

15. RETURN H

```

Figure 4.5: Algorithm Design of Grad-Cam

4.2 External APIs/SDKs

Describe the third-party APIs/SDKs used in the project implementation in the following table.

API / SDK	Version	Description	Purpose of Usage	Specific Function/Class Used
PyTorch	2.x	Deep learning framework	Model training and inference	<code>torch.load()</code> , <code>torch.tensor</code>
Hugging Face Transformers	4.x	NLP library	Loading TinyLLaMA and other language models	<code>AutoModelForCausalLM.from_pretrained</code>
FAISS	1.7	Similarity search library	Storing and searching embeddings	<code>faiss.IndexFlatL2.index.search()</code>
OpenCV (cv2)	4.x	Computer vision library	Preprocessing input images	<code>cv2.GaussianBlur</code> , <code>cv2.threshold</code>
Tesseract OCR	5.x	Optical Character Recognition engine	Extracting text from images	<code>pytesseract.image_to_string</code>
Flask	3.x	Web microframework	Backend API for model serving	<code>Flask()</code> , <code>@app.route</code>
Sentence Transformers	2.x	Embedding library	Converting text to vector embeddings	<code>SentenceTransformer.encode</code>

4.3 Testing Details

Once the system has been successfully developed, testing has to be performed to ensure that the system working as intended.

4.3.1 Unit Testing

Each unit test is designed to test a specific function or method independently from other components, helping to identify issues directly related to the functionality being tested.

UnitTestingTable					
Algorithm / Component	Test Case ID	Test Description	Input Data	Expected Outcome	Status
1. Multimodal RAG	RAG_01	Embedding Fusion	Image (I) + Question (Q)	Combined vector V has dimension d (e.g., 768) and is normalized.	Pass
	RAG_02	Context Retrieval	Query Vector (V)	Returns Top-K (e.g., 5) documents with similarity scores > threshold.	Pass
2. SM-2 Scheduler	SM2_01	Correct Answer Handling	Grade=4, Prev_Interval=1	New Interval > 1 (e.g., 6 days). Ease factor remains stable/increases.	Pass
	SM2_02	Failure Handling	Grade=1, Prev_Interval=20	New Interval resets to 1 day. Ease factor decreases.	Pass
3. Anonymization	ANON_01	Text Detection	X-Ray with burned-in name	Returns bounding boxes for text regions.	Pass
	ANON_02	Blur Verification	Bounding Box coords	Pixel variance in box region decreases significantly (indicating blur).	Pass
4. Saliency (Grad-CAM)	EXPL_01	Heatmap Dimensions	Image (224x224)	Heatmap overlay matches input dimensions (224x224).	Pass
	EXPL_02	ROI Alignment	Image with lung opacity	High heatmap values coincide with the opacity region.	Pass
5. Adaptive Quiz	QUIZ_01	Weak Topic Priority	Accuracy: (Pneumonia: 20%, Normal: 90%)	Generated quiz contains >50% questions on Pneumonia.	Pass
	QUIZ_02	Question Fetching	Selected Topic List	Returns valid Question objects (Text + Image + Answer Key) from DB.	Pass

Figure 4.6: Unit Testing for MedVisionAI

Bibliography

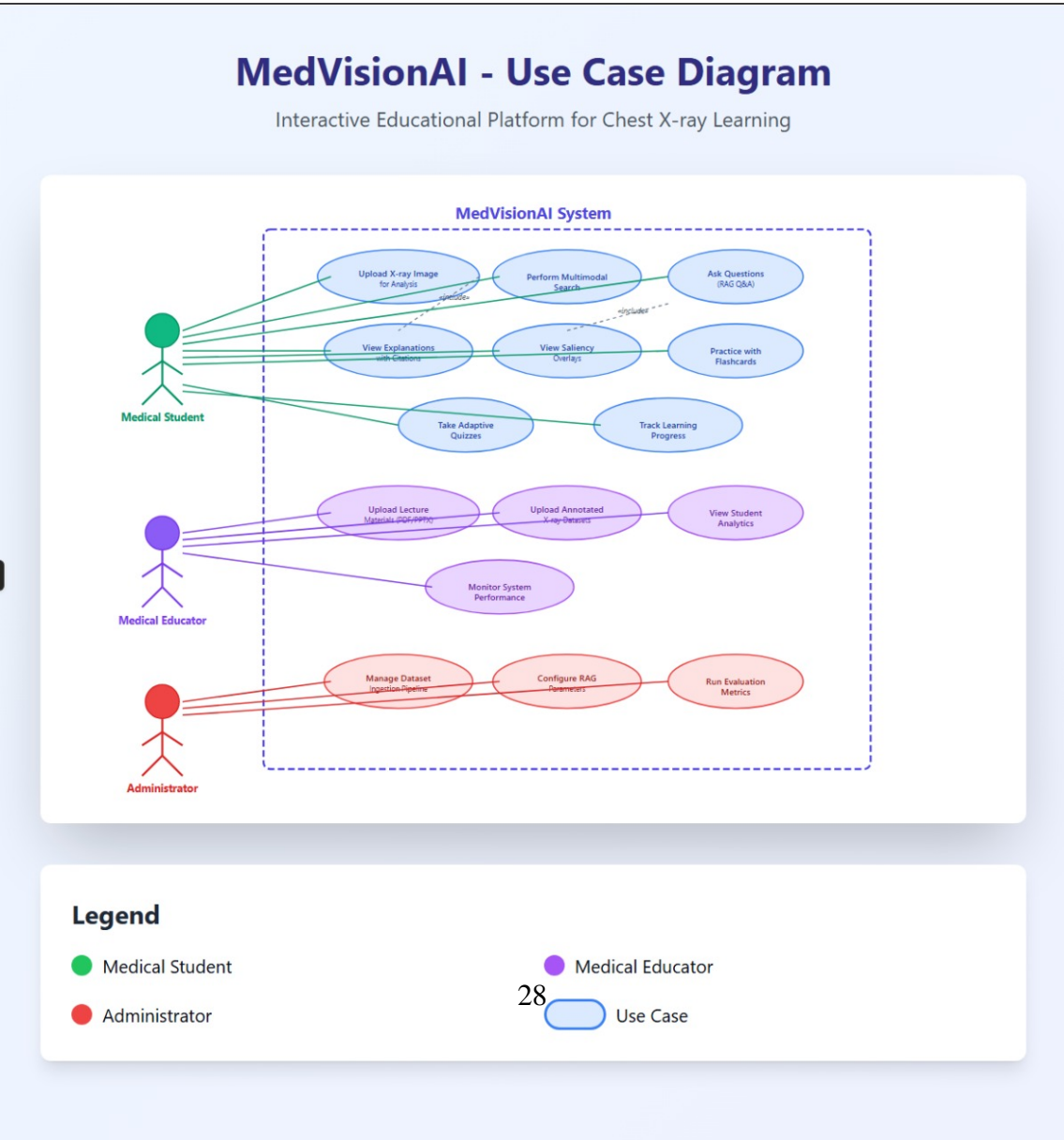
- [1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.
- [2] Edward Hu, Yelong Shen, et al. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [3] Alistair E. W. Johnson, Tom J. Pollard, Nathaniel Shen, et al. Mimic-cxr, a de-identified publicly available database of chest radiographs with free-text reports. *Scientific Data*, 2019.
- [4] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision (clip). *arXiv preprint arXiv:2103.00020*, 2021.
- [5] Facebook AI Research. Faiss: A library for efficient similarity search. <https://github.com/facebookresearch/faiss>, 2025.
- [6] Robin Rombach, Andreas Blattmann, Dominik Lorenz, et al. High-resolution image synthesis with latent diffusion models. *arXiv preprint arXiv:2112.10752*, 2021.
- [7] Xiaosong Wang, Yifan Peng, Le Lu, Zhiyong Lu, Mohammadhadi Bagheri, and Ronald M. Summers. Chestx-ray8: Hospital-scale chest x-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

Appendix A

Appendices

A.1 Appendix A

A.1.1 Use Case Diagram (MedVisionAI)



A.1.2 Detail Use Case

ID	MedVisAI System – Detailed Use Case Specification
UC-001	Displaying elements in a changeable view.
Ingest & Process Documents	The elements are shown in a log window and can be filtered, in a user-selectable way.
Goal	Displaying Process Documents
Preconditions	User is logged in as Instructor/Admin
Goal	The correct elements (filter sorting) are displayed.
Success End Condition	Elements not matching criteria are displayed.
Trigger	User clicks on a button
Success End Condition	The list of elements is updated (refreshes) on the screen
Stakeholder	User.
Actor	1. The list of all elements matching default criteria are shown. 2. System performs OCR (PPTX/imageCVP/images) 3. System processes in (OpenCV) 4. User changes the Filter button. 5. Generating Transformations in (SQL/JSON DB) 6. The user presses a button in FAISS index/SQL/JSON DB 7. Clear the filters.
Perform Multimodal Retrieval & Q.A	What is the best way to find the best results, the search results can be changed the position. The search results for all different columns are at the table result is shown.
Includes	PharWeekly
Alternativity of Use	Login
Take & Adhere to Special Requirements	Login
Assumptions	None
Notes and Issues	None

Figure A.2: Detail Use Case

A.1.3 Event-Response Table for MedVisionAI

Event	System State	System Response
User upload Lecture Slides (PDF/PPT)	Start IngestionPipeline (OCR, Text Extraction as authenticated as Instructor/Admin.	Start Ingestion Pipeline, Raw Extraction in transaction timer as Instructor/Admin.
User upload splerafs tury (Text + Optional Image)	Generate text embeddings Store (D3)	1. Start FAISS Vector green.
User submits Query	RAGEngine is active.	1. Do nothing.
Intessubmits Query	Cross-traffic signal is green.	1. Retrieve top-k search multindool on FAISS Index documents.
User starts Flashcards Session	User dual antanl starcin on LLM	Generate promptStart nest LLM Reidence for Top-k Overlay (pactivaited)
User starts Flashcards hussion	Query FlashcardManager for due hra prodestiont.	Display Answer, Evidence, RAG Annerance, Saliensy Overlay (if (if anryy))
User rates Flashcards Session	Display FlashcardManager for SM-2 schetuten.	Do nothing.
Quiz starts Quiz Session	Pedestrian sign tedic dord for Manswser (SM-2 scheduled, Reenead Progress (D4)	1. Update User User's Expluestionn (SM-2 Quodie (D4)
Quiz Starts Quiz Answer	Save Submits Quiz Answer	Save Use mid Score & Summary
Instructor/Admin uploadas CXR Datssets	1. Ste Process CXR Images De-ididicied Sluslc Inge Store). Dativede (D3)	1. Eitz PuosEnstized Irsage Anoyminize, Caption, Reer acus User Progss (D4)
Instructor/Admin uploadad Public CXR Datssets (De-lified Image Store () User is anstheniciated.	Start Pess CXR Images Anoyminize, Caption, tore). Genate io ased hisere from FAISS Vector IndS)	IRetreve user study int sohe Topic-daited Image, Salinescy). Uptate FAISS Vector Ind3)

Figure A.3: Event Response Table

A.1.4 Story Board Example For MedVisionAI

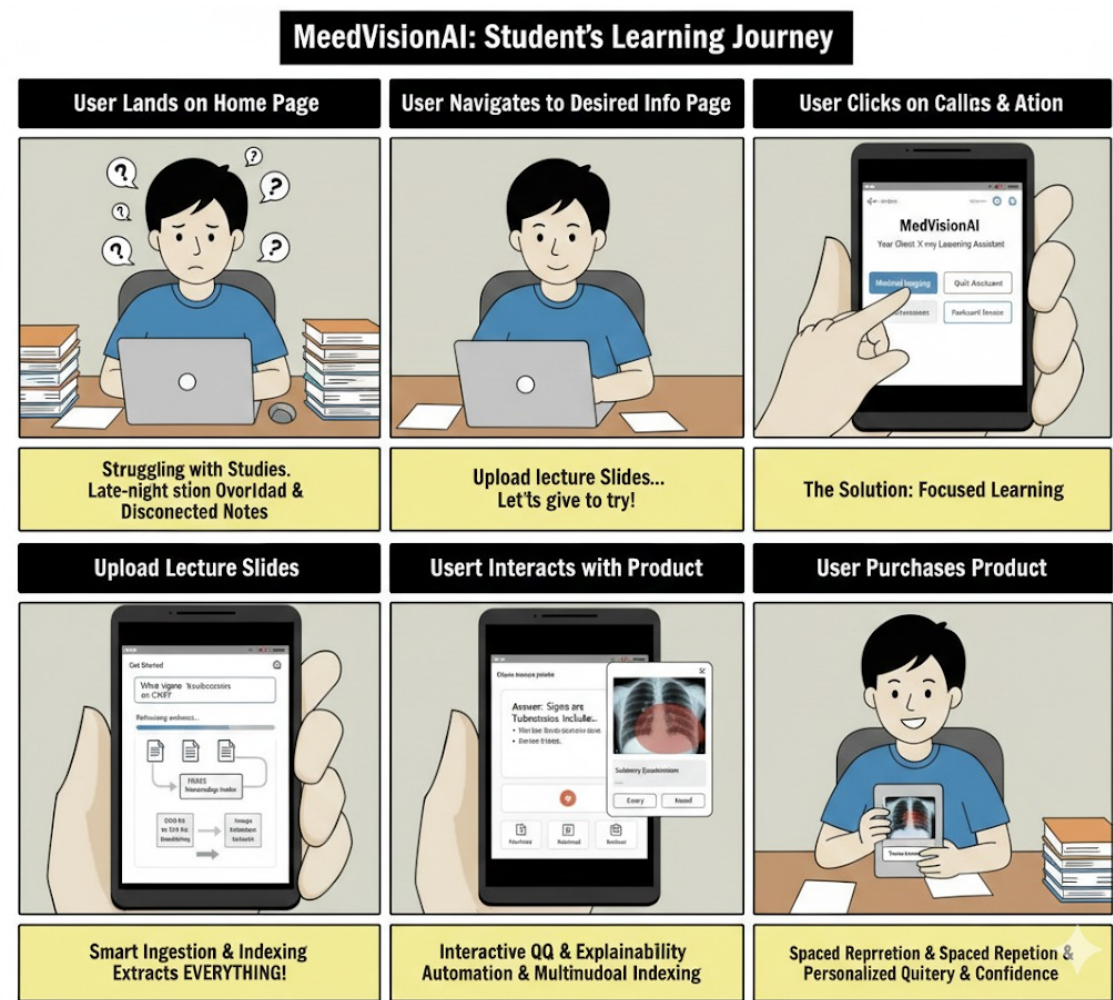


Figure A.4: Story Board

A.2 Appendix B

A.2.1 Domain Model for MedVisionAI

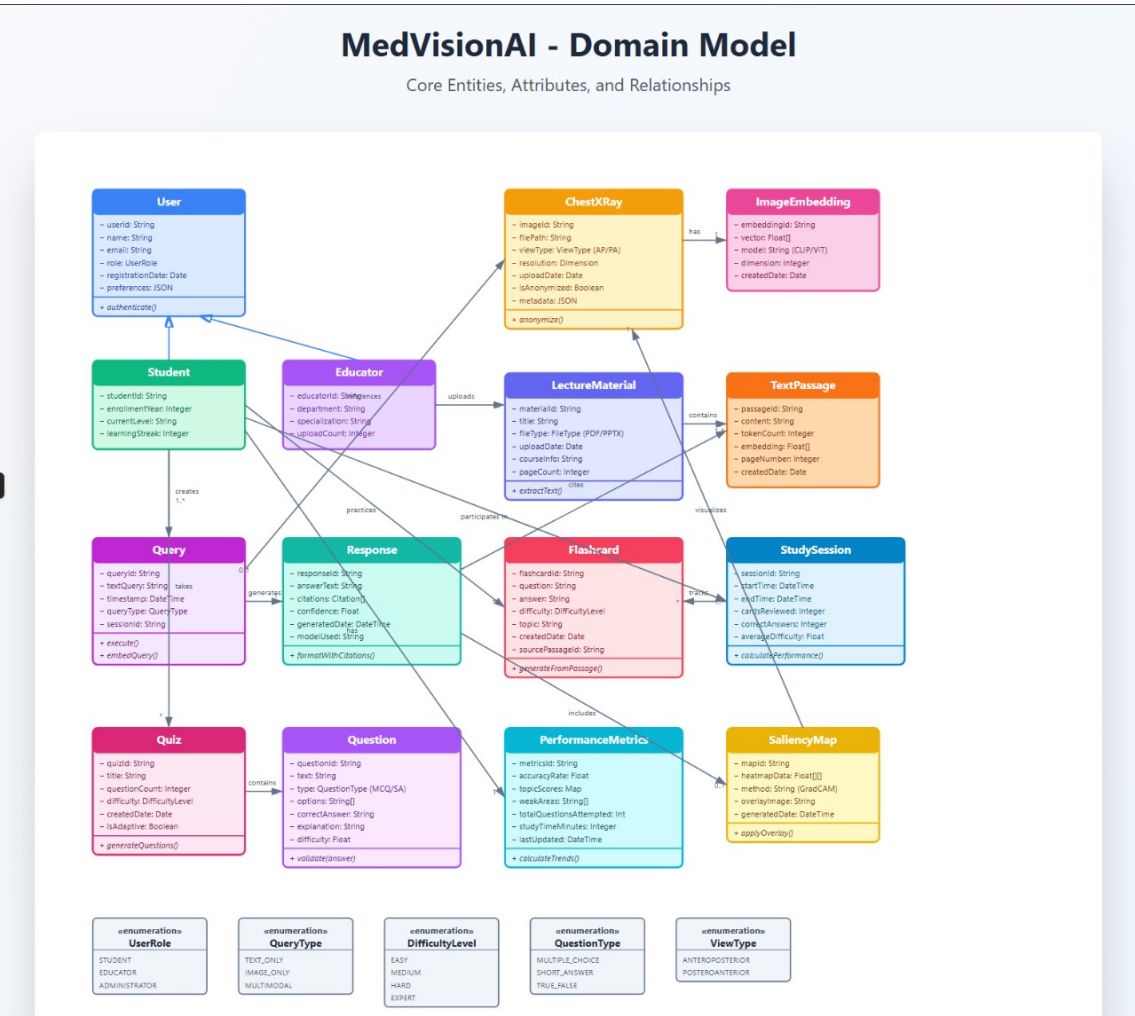


Figure A.5: Domain Model

A.3 Appendix C

A.3.1 Box And Line For MedVisionAI

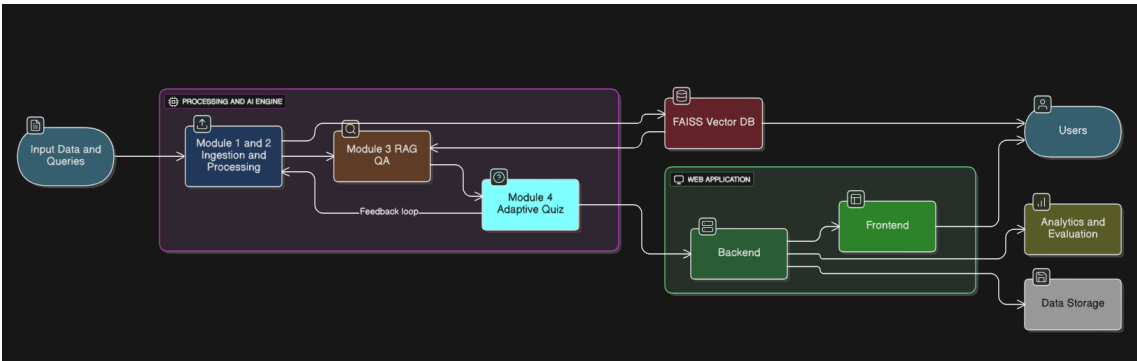


Figure A.6: Box and Line Diagram For MedVisionAI

A.3.2 Architecture Pattern for MedVisionAI

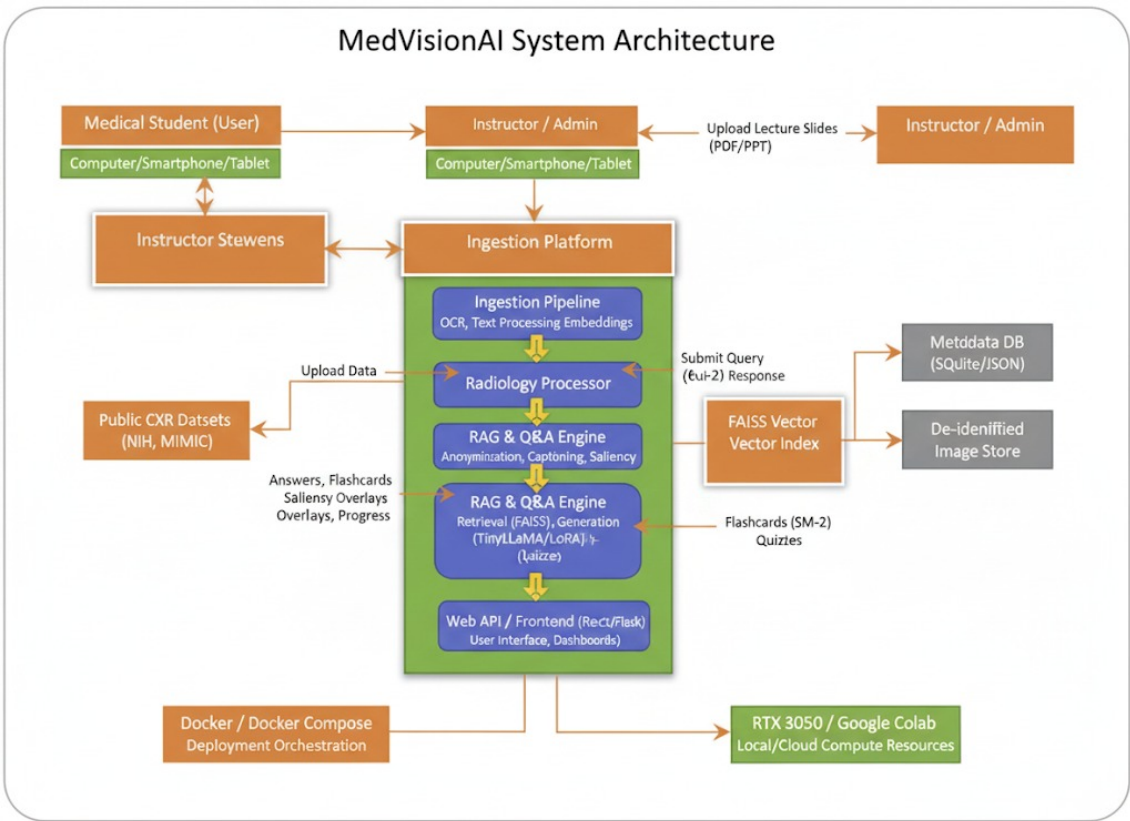


Figure A.7: Architecture Pattern For MedVisionAI

A.4 Appendix D

A.4.1 Activity Diagram

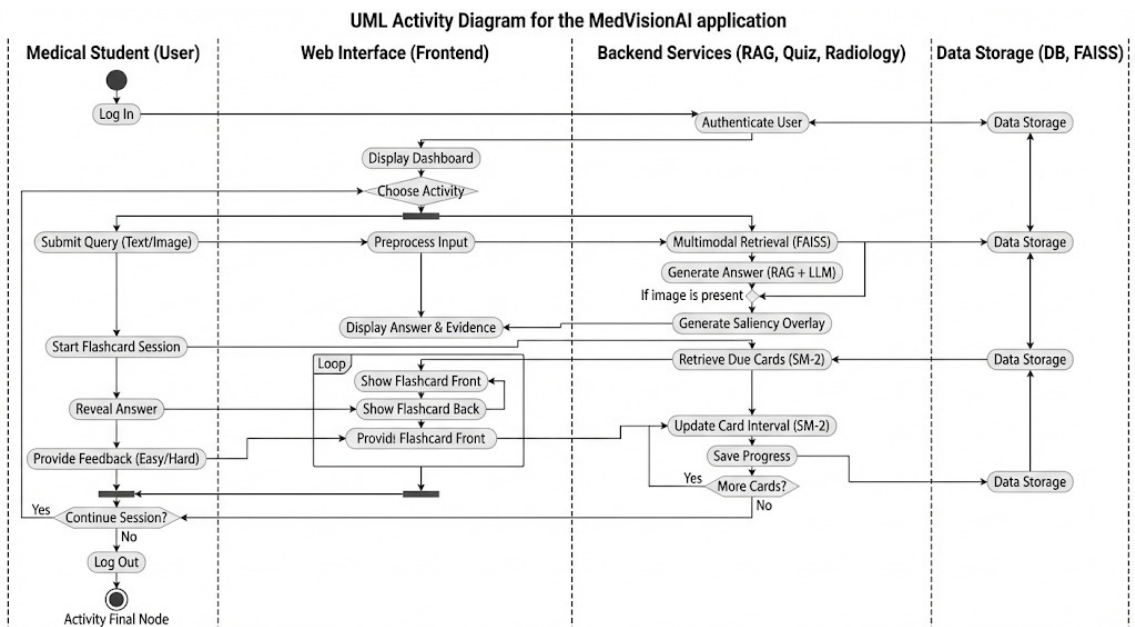


Figure A.8: Activity Diagram For MedVisionAI

A.4.2 Class Diagram

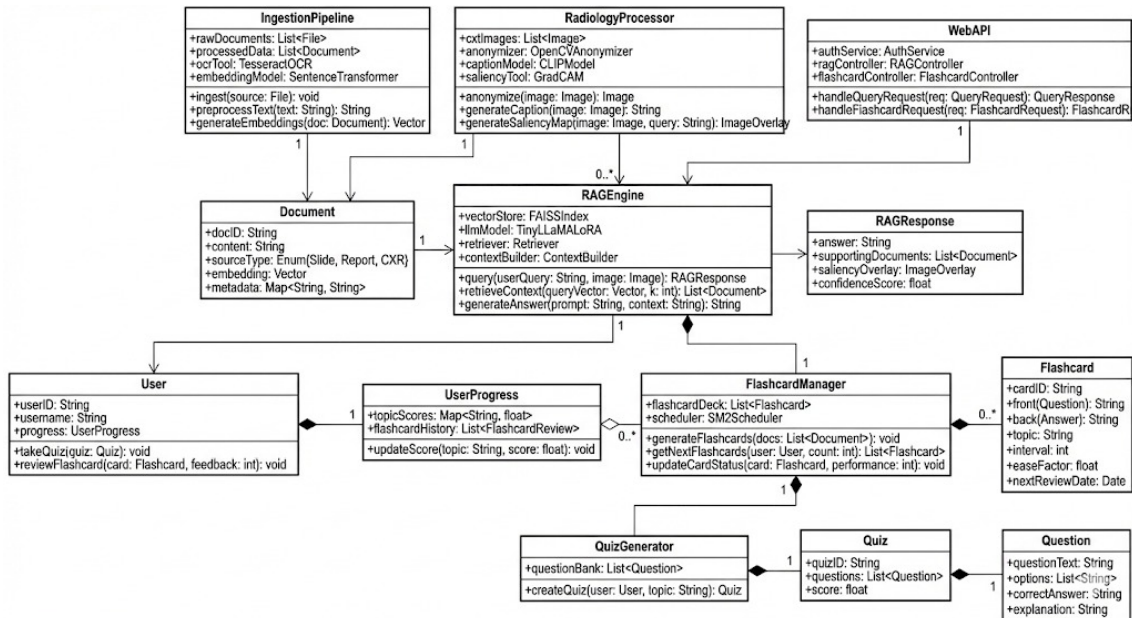


Figure A.9: Class Diagram For MedVisionAI

A.4.3 Sequence Diagram

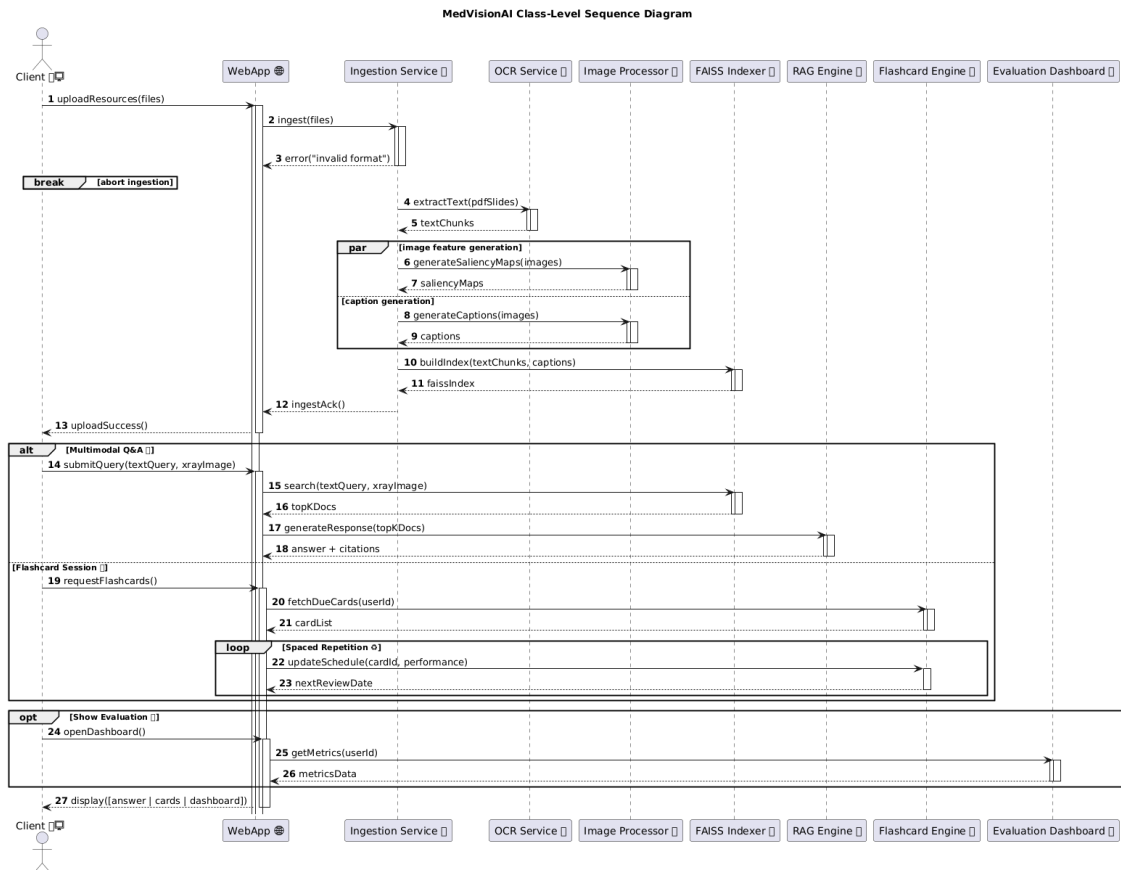


Figure A.10: Sequence Diagram For MedVisionAI

A.4.4 State Transition Diagram

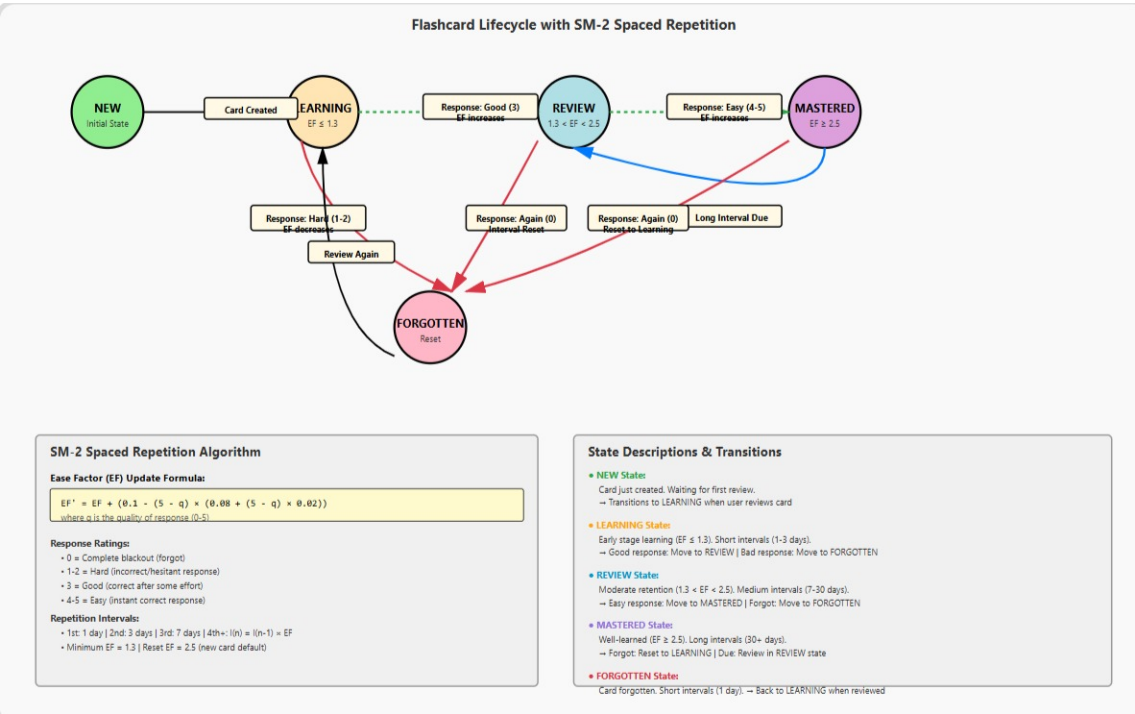


Figure A.11: State Transition Diagram For Online Shopping Application

A.4.5 Data Flow Diagram

